Technical report

Reducing Newton iterations for Navier Stokes using Machine Learning - COMSOL

WI5215: Internship Applied Mathematics

Giovanni Matarazzo

Delft University of Technology



TU Delft

Technical report

Reducing Newton iterations for Navier Stokes using Machine Learning - COMSOL

by

Giovanni Matarazzo

Project Duration: August, 2026 - November, 2026
Faculty: Applied Mathematics
Supervisors: Tycho van Noorden and Lisanne Rens

Cover: Canadarm 2 Robotic Arm Grapples SpaceX Dragon by NASA under CC BY-NC 2.0 (Modified)
Style: TU Delft Report Style, with modifications by Daan Zwaneveld

TU Delft

Contents

Nomenclature ii

1 Introduction 1

2 Numerical formulation of the problem 3
2.1 Governing Equations . . . 3
2.2 Temporal Discretization . . . 3
2.3 Nonlinear Solution Procedure . . . 4
2.4 Imposing initial guess in a time-dependent problem . . . 4
2.5 Data for a graph neural network . . . 5
2.6 Initial transient behaviour . . . 6
2.7 Evaluation framework . . . 6
2.8 General pipeline of the project . . . 6

3 Data generation 8
3.1 2D Channel with Cylinder . . . 8
3.2 Generalization Reynolds number . . . 9
3.3 Generalization different geometries . . . 9
3.4 Data features for the graph (nodes and edges) . . . 10
3.4.1 Additional features . . . 11
3.5 Graph Dataset Construction . . . 11
3.6 Normalization . . . 12

4 Graph Neural Network architecture 14
4.1 MeshGraphNet - velocity architecture . . . 14
4.2 Bi-stride Multi-Scale GNN - Pressure Architecture . . . 15
4.3 Initialization of the network parameters . . . 17
4.4 Loss functions . . . 18
4.4.1 Physics-based residual loss . . . 18
4.4.2 Jacobian scaled loss . . . 19
4.4.3 Top-k loss . . . 20
4.5 Hard enforcement boundary condition . . . 20

5 Results 22
5.1 Results on Reynolds numbers dataset . . . 22
5.1.1 Jacobian scaled loss results . . . 23
5.2 Results on different geometries dataset . . . 24
5.2.1 Jacobian scaled loss results . . . 25
5.3 Hyperparameter optimization results . . . 26

6 Conclusion 29

References 31

A COMSOL configuration details 33
A.0.1 Time-Dependent Solver . . . 33

B Configuration Hyperparameters 35

Nomenclature

Abbreviations

Abbreviation	Definition
BC	Boundary Condition
BDF	Backward Differentiation Formula
BSMS	Bi-Stride Multi-Scale Graph Neural Network
FEM	Finite Element Method
GCN	Graph Convolutional Network
GNN	Graph Neural Network
MGN	MeshGraphNet
MSE	Mean Squared Error
PDE	Partial Differential Equation
RMSE	Root Mean Squared Error
WLSQ	Weighted Least-Squares

Symbols

Symbol	Definition	Unit
D	Cylinder diameter	[m]
$\mathbf{f}$	Body force per unit volume	[N/m^3]
H	Channel height	[m]
h	Time-step size	[s]
J	Jacobian matrix of the discretized nonlinear system	[-]
L	Characteristic length scale	[m]
N	Number of mesh/graph nodes	[-]
p	Pressure	[Pa]
p^*	Dimensionless pressure	[-]
R	Cylinder radius	[m]
$\mathbf{R}$	Residual vector of the discretized nonlinear system	[-]
Re	Reynolds number	[-]
t	Time	[s]
Δt	Time-step size	[s]
u	Velocity component in the x-direction	[m/s]
u^*	Dimensionless velocity component in the x-direction	[-]
v	Velocity component in the y-direction	[m/s]
v^*	Dimensionless velocity component in the y-direction	[-]
$\mathbf{u}$	Velocity vector	[m/s]
U_∞	Reference velocity used for nondimensionalization	[m/s]
U_{mean}	Mean inlet velocity	[m/s]
$\mathbf{x}$	Discrete flow-state vector	[-]
μ	Dynamic viscosity	[Pa s]
ν	Kinematic viscosity	[m^2/s]
ρ	Fluid density	[kg/m^3]

1 Introduction

Computational Fluid Dynamics (CFD) is widely used to analyse and predict fluid flow behaviour in engineering and scientific applications. In many CFD simulations, the equations describing fluid flow are discretised using the Finite Element Method (FEM). The Finite Element Method is implemented in various software packages, of which COMSOL Multiphysics [4] is an example.

In this work, we consider a time-dependent laminar flow problem which is described by the Navier Stokes equations. When solving these equations numerically, convergence difficulties may arise, because of their inherent nonlinearity. In particular, for a time dependent simulation, at every time-step the Finite Element solver needs to solve a nonlinear system of algebraic equations, which is commonly solved using Newton's method [9], which, under suitable regularity assumptions and for an initial guess sufficiently close to the solution, exhibits locally quadratic convergence [22]. Therefore, it is crucial that our initial guess of the nonlinear system is "close enough" to the real solution to provide a sufficiently fast convergence and, more in general, to actually converge.

The initial guess that is provided in COMSOL to the nonlinear system for the new time-step is usually the solution of the previous timestep [4]. However, the previous time-step solution may not always provide a sufficiently accurate initial guess, potentially increasing the number of nonlinear iterations or, in more difficult cases, preventing convergence. Because of this, it might be helpful to improve the first guess provided to the nonlinear solver.

Graph neural networks (GNNs) have emerged as a suitable architecture to handle mesh-based simulation problems [23], modeling, for example, elements as nodes and their connectivity edges as graph edges, allowing the network to capture spatial relationships naturally. For this reason and based on the positive results in [12] with Graph Neural Networks to predict the optimal CFL number for stationary Navier Stokes, GNNs seem a promising hybrid solution to reduce the number of Newton iterations in COMSOL for Navier Stokes.

The main objective of this work is therefore to investigate whether a Graph Neural Network can provide improved initial guesses for the nonlinear solution of time-dependent incompressible Navier–Stokes problems, thereby reducing the computational effort required by the Newton solver. Can a GNN predict the flow state at the next time-step more accurately than the standard initialization based on the previous time-step solution? Does an improvement in the predicted initial state translate into a reduction in the number of Newton iterations required for convergence? Can training objectives that account for the nonlinear FEM system produce initial guesses that are more effective for Newton convergence than conventional prediction losses?

The rest of the report is organized as follows. The numerical formulation of the problem is given in Chapter 2. Chapter 3 then describes the domains on which the problem is studied and explains the process of data generation. Chapter 4 introduces the neural networks with their architectures and training procedures. Chapter 5 discusses the results found and the model performance. Chapter 6 finally concludes the findings and gives suggestions for further research. In Appendix A can be found

more details about the implementation code structure, in Appendix B COMSOL configuration details, in Appendix C configuration details about the hyperparameter used in the best performing configuration.

2

Numerical formulation of the problem

2.1. Governing Equations

In this work, we consider a two-dimensional, time-dependent, incompressible laminar flow problem. The fluid dynamics are governed by the incompressible Navier–Stokes equations,

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u}\right) = -\nabla p + \mu\nabla^2\mathbf{u} + \mathbf{f}, \tag{2.1}$$

$$\nabla\cdot\mathbf{u} = 0, \tag{2.2}$$

where $\mathbf{u} = (u, v)$ denotes the velocity field, p the pressure, ρ the fluid density, μ the dynamic viscosity, and $\mathbf{f}$ a body force acting on the fluid.

2.2. Temporal Discretization

The incompressible Navier–Stokes equations contain the time derivative of the velocity field. In order to numerically advance the solution from a known state at time (t_n) to the new state at (t_{n+1}), this derivative must be discretized in time.

A general Backward Differentiation Formula (BDF) approximates the time derivative at (t_{n+1}) using the unknown solution at the new timestep and a number of previously computed solutions and, in the variable-step case, on the timestep sizes.

For example, using the first-order Backward Euler scheme (BDF1),

$$\left.\frac{\partial \mathbf{u}}{\partial t}\right|_{t_{n+1}} \approx \frac{\mathbf{u}_{n+1} - \mathbf{u}_n}{\Delta t}.$$

(2.3)

Substitution into the momentum equation gives

$$\rho\frac{\mathbf{u}_{n+1} - \mathbf{u}_n}{\Delta t} + \rho\left(\mathbf{u}_{n+1}\cdot\nabla\right)\mathbf{u}_{n+1} = -\nabla p_{n+1} + \mu\nabla^2\mathbf{u}_{n+1} + \mathbf{f}_{n+1}. \tag{2.4}$$

Together with the incompressibility constraint,

$$\nabla \cdot \mathbf{u}_{n+1} = 0, \tag{2.5}$$

this defines the problem that must be solved at the new timestep. The quantities from previous timesteps are already known, while $\mathbf{u}_{n+1}$ and p_{n+1} are the unknowns.

2.3. Nonlinear Solution Procedure

COMSOL Multiphysics is based on the Finite Element Method (FEM). Therefore, the solution of the nonlinear algebraic equations is obtained after the discretization of the weak form of the governing equations [29].

In practice, the discretized problem consists of finding the solution ($\mathbf{x}_{n+1}$) such that

$$\mathbf{R}_{n+1}\left(\mathbf{x}_{n+1}\right) = \mathbf{0}, \tag{2.6}$$

where ($\mathbf{R}_{n+1}(\mathbf{x})$) represents the nonlinear residual of the discretized finite element system for the $(n+1)$-th timestep and the state vector is given by

$$\mathbf{x}_{n+1} = \begin{bmatrix} \mathbf{u}_{n+1} \\ \mathbf{p}_{n+1} \end{bmatrix}. \tag{2.7}$$

A classical method used to solve such a nonlinear system is Newton's method [7]. At the (k)-th nonlinear iteration, the following linearized system is solved:

$$\mathbf{J}_{n+1}\left(\mathbf{x}_{n+1}^{(k)}\right)\Delta\mathbf{x}_{n+1}^{(k)} = -\mathbf{R}_{n+1}\left(\mathbf{x}_{n+1}^{(k)}\right), \tag{2.8}$$

where

$$\mathbf{J}_{n+1}(\mathbf{x}) = \frac{\partial \mathbf{R}_{n+1}(\mathbf{x})}{\partial \mathbf{x}} \tag{2.9}$$

is the Jacobian matrix of the nonlinear residual and $\mathbf{x}_{n+1}^{(k)}$ is the approximation of the solution to Equation 2.6.

The solution is then updated according to

$$\mathbf{x}_{n+1}^{(k+1)} = \mathbf{x}_{n+1}^{(k)} + \Delta\mathbf{x}_{n+1}^{(k)}. \tag{2.10}$$

This procedure is repeated until the convergence criteria are satisfied, which can depend, for example, on the size of the update or the maximum allowed number of nonlinear iterations.

For this reason, the choice of the initial guess ($\mathbf{x}_{n+1}^{(1)}$) is particularly important. The initial guess should be sufficiently close to the converged solution to ensure convergence and, at the same time, to reduce the number of nonlinear iterations required to reach convergence. In this work, the objective is therefore to replace the standard initial guess with the prediction of the GNN, while keeping the nonlinear problem being solved unchanged. This method has been used to temporally discretize our equations in the time-dependent simulations, in particular with BDF2 [14].

2.4. Imposing initial guess in a time-dependent problem

During the development of the model, it was necessary to substitute the initial guess that COMSOL Multiphysics would use automatically at each timestep (hence the solution of the previous timestep) with the result of the prediction of our GNN architecture.

In COMSOL, it is not possible for a time-dependent problem to impose directly the initial guess for the time dependent solver at each timestep [4], hence we solved a Navier Stokes stationary problem

introducing the time dependent component as a forcing component using Backward Differentiation formulas (BDF):

$$\rho(\mathbf{u}\cdot\nabla)\mathbf{u} - \mu\nabla^2\mathbf{u} = -\nabla p + \mathbf{f}_{\text{stationary}}, \tag{2.11}$$

$$\rho\nabla\cdot\mathbf{u} = 0. \tag{2.12}$$

where: $\mathbf{f}_{\text{stationary}} = \mathbf{f}_{\text{time dependent}} + \mathbf{BDF}_{\text{term}}$.

In particular, we have implemented BDF1 and BDF2 [14] formula (according to the discretization used by the time dependent solver) with variable step size, hence the momentum equation when discretized with BDF1 or BDF2 is respectively:

$$\rho\left(\mathbf{u}^{n+1}\cdot\nabla\right)\mathbf{u}^{n+1} - \mu\nabla^2\mathbf{u}^{n+1} = -\nabla p^{n+1} + \mathbf{f}^{n+1} - \rho\frac{\mathbf{u}^{n+1}-\mathbf{u}^{n}}{h_n}, \qquad h_n = t_{n+1} - t_n. \tag{2.13}$$

$$\rho\left(\mathbf{u}^{n+1}\cdot\nabla\right)\mathbf{u}^{n+1} - \mu\nabla^2\mathbf{u}^{n+1} = -\nabla p^{n+1} + \mathbf{f}^{n+1} - \rho\frac{1}{h_n}\left[\frac{1+2w}{1+w}\mathbf{u}^{n+1} - (1+w)\mathbf{u}^{n} + \frac{w^2}{1+w}\mathbf{u}^{n-1}\right],$$
$$w = \frac{h_n}{h_{n-1}}, \quad h_n = t_{n+1} - t_n. \tag{2.14}$$

Therefore, as shown in the previous section, by means of this implementation the stationary formulation reproduces the time-discretized governing equations at a given timestep. It is important to notice that actually COMSOL uses additional stabilization terms such as Streamline Diffusion and Crosswind Diffusion for the time dependent problem [4]. We have added the Streamline stabilization term to the forcing terms of the stationary solver to keep the stationary solver solutions similar to those of the time dependent problem and it has been measured numerically that the difference between the time-dependent solution and the stationary one with the above mentioned forcing terms differ by a relatively small quantity (e.g., with a Root Mean Squared Error (RMSE) over the mesh on the order of 10^{-8} or even smaller for both velocity and pressure).

The performance of the prediction provided by the GNN was measured in the following way. Firstly a time-dependent simulation was run in COMSOL, saving the results for each timestep to train the GNN. Secondly, for each timestep the stationary solver was solved for the expression 2.11, 2.12 and two independent simulations were run changing just the initial condition. The first one represented the standard baseline and was initialized with the solution of the previous timestep (taken from the time-dependent simulation solution), the second one was the result of the GNN prediction. The performance of the GNN is measured by comparing the number of Newton iterations for both simulations.

2.5. Data for a graph neural network

Given the structure of a mesh, because of the flexibility and adaptive nature of graph neural networks, it seems natural to represent a mesh using a graph. In previous work [12], the graph has been modeled in the following way: centroids elements represent graph nodes and there is an edge among two elements that have at least one adjacent side. The reason for this choice was that the flow is described on a full element, so taking its centroid is much more representative than taking corners of elements [12]. Nevertheless, the objective of this project is different and we want to predict an optimal initial guess for the Newton solver for the next time step. Because of that, if we would have used the centroids as the nodes of the graph, we would have needed some kind of projection to find the values of the predicted quantities on the Degrees of Freedom (DOFs) of the model.

Because of this reason, we used the corners of the mesh as graph nodes and the edges as graph edges. Furthermore, we used a P1-P1 discretization [9], and as a result the graph nodes correspond to the DOFs of the FEM system, hence there is a direct correspondence among the graph prediction (node level prediction) of our GNN and the DOFs that COMSOL uses to solve the simulation. The connectivity of the graph is described by an adjacency list of pairs (i, j) containing nodes indices. Hence each pair in the list represents an edge from node i to node j. Given that the graph is clearly undirected, each pair appears twice in the adjacency list.

2.6. Initial transient behaviour

During the simulations, an initial phase has been observed in the first few timesteps, during which the pressure field undergoes significant variations throughout the entire domain. This behaviour arises because, even though a smooth step function is used to introduce the inlet velocity, the simulation still starts from artificially imposed initial and boundary conditions that would generally not occur in a real-world scenario. As a consequence, large variations are observed during the first timesteps, which do not represent the core dynamics of the system.

These initial timesteps exhibit a behaviour that is substantially different from that observed during the remainder of the simulation, for instance in terms of the scale of the pressure variations. For this reason, it was decided to exclude the corresponding initial snapshots from the dataset and instead focus on the dynamics of interest. Including these snapshots could otherwise contaminate the training process with the artificial initial transient and potentially affect the ability of the model to learn the dynamics characterizing the rest of the simulation.

An alternative approach would be to focus specifically on the initial transient dynamics. This could also be relevant for the objective of this work, since the nonlinear solver may require a large number of Newton iterations during these first timesteps. However, investigating this regime would require a substantially different choice of training and evaluation datasets, since the initial transient occurs only during the very first timesteps of each simulation.

Another possible approach would be to use the COMSOL Fully Developed Flow inlet boundary condition [4]. However, this would prevent the inlet velocity values from being known a priori, which would introduce an additional difficulty for the present GNN formulation.

2.7. Evaluation framework

As previously explained in the introduction, the goal of this project is to reduce the number of Newton iterations that COMSOL Multiphysics requires to converge in the time-dependent Navier Stokes problem. Nevertheless, as shown in [21] a supervised learning based on the mean-squared error (MSE) between the prediction of the GNN and the solution of the next timestep does not guarantee that the number of Newton iterations is going to be reduced. The residual $\mathbf{R}(\mathbf{x})$ and the MSE are two different quantities and there is no mathematical guarantee that minimizing one is equivalent to reducing the other. Furthermore, the Newton residual of FEM equations $\mathbf{R}(\mathbf{x})$ can be obtained from COMSOL numerically at each iteration, but its evaluation is external to the automatic-differentiation computational graph used for GNN training, hence it is not possible to use $\mathbf{R}(\mathbf{x})$ as an objective function for our loss function, since it would not be possible to back-propagate the loss of the network in order to update the parameters of the GNN. In conclusion, the evaluation of the GNN performance is not based on how well it performs in terms of its loss function minimization but on how able it is to reduce the number of Newton iterations.

2.8. General pipeline of the project

As observed by [23], in order to predict $u^{n+1}, v^{n+1}, p^{n+1}$ in a time-dependent problem given u^n, v^n, p^n with a GNN, **it is more convenient to predict just the update** $\Delta u^{n+1} = u^{n+1} - u^n, \Delta v^{n+1} = v^{n+1} - v^n, \Delta p^{n+1} = p^{n+1} - p^n$ instead of learning the total field. The figure 2.1 shows schematically what has been done in this project. For a transition from t_n to t_{n+1}, the standard initialization considered in this work is

$$\mathbf{x}^{(0)}_{n+1,\mathrm{std}} = \mathbf{x}_n, \tag{2.15}$$

whereas the GNN-based initialization is given by

$$\mathbf{x}^{(0)}_{n+1,\mathrm{GNN}} = \widehat{\mathbf{x}}_{n+1}. \tag{2.16}$$

Where $\widehat{\mathbf{x}}_{n+1}$ is the prediction of the GNN from $\mathbf{x}_n$ to $\mathbf{x}_{n+1}$.

In both cases, the nonlinear system, boundary conditions, solver settings, and convergence criteria are kept unchanged. Therefore, the only difference between the two solution is the initial guess provided to the Newton solver.

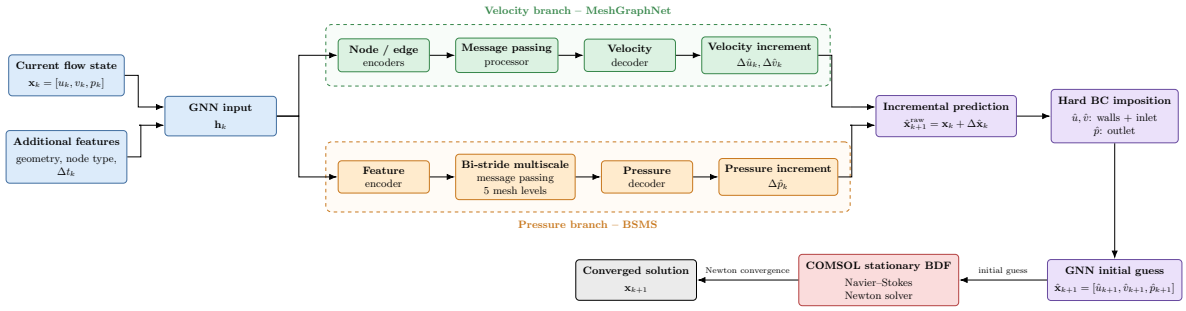


Figure 2.1: General pipeline of the project.

3

Data generation

The Navier–Stokes equations are discretized in COMSOL Multiphysics using the Finite Element Method (FEM). A prescribed parabolic velocity profile is imposed at the inlet, while a pressure condition is applied at the outlet.

The fluid properties used throughout this work are

$$\rho = 1 \text{ kg/m}^3, \qquad \mu = 1 \times 10^{-3} \text{ Pa s}. \tag{3.1}$$

The flow field is initialized from rest, with the initial conditions

$$u(\mathbf{x}, 0) = 0, \qquad v(\mathbf{x}, 0) = 0, \qquad p(\mathbf{x}, 0) = 0. \tag{3.2}$$

The flow regime is characterized by the Reynolds number,

$$Re = \frac{\rho U L}{\mu} = \frac{U L}{\nu}, \tag{3.3}$$

where U is a characteristic velocity and L is a characteristic length scale.

3.1. 2D Channel with Cylinder

The first geometry used is of a two-dimensional rectangular channel containing a slightly asymmetrically positioned cylinder. This configuration represents a classical benchmark problem for incompressible flow [27]. The slight asymmetry of the geometry allows the development of an unsteady flow regime, preventing the solution from converging to a stationary state and therefore preserving the transient nature of the flow.



(a) Inlet boundary. **(b)** Outlet boundary. **(c)** No-slip wall boundaries.

Figure 3.1: In blue lines the boundary conditions of the two-dimensional channel with cylinder: (a) inlet boundary, (b) outlet boundary, and (c) no-slip wall boundaries.

The inflow boundary condition is prescribed using a parabolic velocity profile of the form

$$\mathbf{u}_{\text{in}}(y,t) = \begin{bmatrix} u_{\text{in}}(y,t) \\ v_{\text{in}}(y,t) \end{bmatrix} = \begin{bmatrix} 6U_{\text{mean}} \dfrac{y(H-y)}{H^2} \ \text{step}(t) \\ 0 \end{bmatrix}. \tag{3.4}$$

Here, $\text{step}(t)$ denotes the step function defined in COMSOL with C^2 continuity, which is used to smoothly ramp the inlet velocity from zero to its final value[4].

A no-slip boundary condition is imposed on both the channel walls and the cylinder surface [4]:

$$\mathbf{u} = \mathbf{0}. \tag{3.5}$$

The main geometric parameters of the domain are

$$L = 2.2 \text{ m}, \qquad R = 0.05 \text{ m}, \qquad H = 0.41 \text{ m}, \tag{3.6}$$

where L denotes the channel length, R the cylinder radius, and H the channel height. The coordinates of the low left corner are $(x, y) = (0, 0)$ and the coordinates of the center of the cylinder are $(x, y) = (4R, 4R)$

3.2. Generalization Reynolds number

Rather than considering a single Reynolds number, the first dataset used was constructed to evaluate how well the GNN is able to generalize on different Reynolds number configurations, keeping the same geometry configuration of Section 3.1. Specifically, simulations are performed for

$$Re = \{40,\ 60,\ 80,\ 100,\ 120,\ 140,\ 160,\ 180,\ 200\}. \tag{3.7}$$

$$Re = \frac{\rho U_{\text{mean}} 2R}{\mu} \tag{3.8}$$

and respectively:

$$U_{\text{mean}} = \{0.4,\ 0.6,\ 0.8,\ 1.0,\ 1.2,\ 1.4,\ 1.6,\ 1.8,\ 2.0\} m/s. \tag{3.9}$$

U_{mean} is considered as the characteristic velocity since it is the average velocity in the $x-$ direction across the channel as can be seen :

$$\frac{\int_0^H 6U_{\text{mean}} \frac{y(H-y)}{H^2}\, dy}{H} = \frac{1}{H} \frac{6U_{\text{mean}}}{H^2} \frac{H^3}{6} = U_{\text{mean}}. \tag{3.10}$$

Varying the Reynolds number allows the model to be trained and evaluated across different flow conditions. At the lower Reynolds numbers, the flow remains relatively steady and symmetric, whereas increasing the Reynolds number leads to unsteady vortex shedding and the formation of the characteristic von Kármán vortex street downstream of the cylinder [11].

The mesh that has been used is a Free Triangular matrix calibrated for Fluid Dynamics, with a maximum element size of 0.0144 m.

3.3. Generalization different geometries

Rather than considering a single geometry, the second dataset used was constructed to evaluate how well the GNN is able to generalize on different geometry configurations. Following the same approach adopted in [28], five different cylinder configurations used in the training phase were constructed. For each configuration, the coordinates of the center of the cylinder (x_c, y_c) and its radius R were independently sampled from the following uniform distributions:

$$x_c \sim \mathcal{U}(0.15,\ 0.50) \text{ m}, \qquad y_c \sim \mathcal{U}(0.10,\ 0.30) \text{ m}, \qquad R \sim \mathcal{U}(0.02,\ 0.08) \text{ m}. \tag{3.11}$$

The resulting values for the five configurations are

$$x_c = [0.46436507,\ 0.19641516,\ 0.43322998,\ 0.21152758,\ 0.40035719]\ \mathrm{m}, \tag{3.12}$$
$$y_c = [0.10729651,\ 0.17371890,\ 0.23986040,\ 0.14900044,\ 0.27752236]\ \mathrm{m}, \tag{3.13}$$
$$R = [0.04016393,\ 0.06368265,\ 0.07693768,\ 0.02177998,\ 0.03850868]\ \mathrm{m}. \tag{3.14}$$

To evaluate the generalization capabilities of the GNN in terms of geometry, based on [28] the results have been tested on the following unseen geometries in the training phase (which can be seen in Figure 3.2:

- cylinder_tri_quad
- cylinder_stretch
- mixed_all_quad



(a) Cylinder tri quad **(b)** Cylinder stretch **(c)** Mixed all

Figure 3.2: Geometries tested for generalization

The low left corner coordinates are always $(x, y) = (0, 0)$ for all these configurations, the lenght of the channel L and the height of the channel H are the same as before.

For the cylinder tri quad configuration, the position of the low corner of the squared obstacle is $(x, y) = (0.3, 0.15)\mathrm{m}$, the length of its side is $l = 0.1\mathrm{m}$. For the cylinder stretch configuration, the coordinates of the center of the ellipse are $(x, y) = (0.25, 0.25)\mathrm{m}$ the minor semi-axis length is $a = 0.05\mathrm{m}$ the major semi-axis length is $b = 0.1\mathrm{m}$. For the mixed all configuration, the low left corner coordinates of the rectangle obstacle are: $(x, y) = (0.2, 0.175)\mathrm{m}$ its height and width are respectively $0.1\mathrm{m}$ and $0.05\mathrm{m}$, the low left corner coordinates of the triangle obstacle are $(x, y) = (0.5, 0.1)\mathrm{m}$ its base and height are $0.2\mathrm{m}$ and $0.1\mathrm{m}$.

3.4. Data features for the graph (nodes and edges)

The nodes features that are extracted from the mesh (hence the features that are provided to each node) for every node i in order to train the network to predict $u_i^{n+1}, v_i^{n+1}, p_i^{n+1}$ are the following:

- u_i^n, v_i^n, p_i^n
- $\Delta t_{n+1} = t_{n+1} - t_n$
- The distance from wall, inlet and outlet boundaries by means of COMSOL Wall Distance Interface, which is using a modified *eikonal equation* [4].
- Three binary indicator variables identifying whether node i belongs to a specific boundary type:

$$I_{i,j} = \begin{cases} 1, & \text{if node } i \in \Gamma_j, \\ 0, & \text{otherwise}, \end{cases} \qquad j \in \{\text{inlet, outlet, wall, interior}\}, \tag{3.15}$$

where Γ_j denotes the set of nodes belonging to boundary type j.

The edge features that are extracted from the mesh for every edge (i, j) are the followings:

For each edge connecting nodes i and j, the relative spatial position between the two nodes is used to define the edge features. The edge feature vector is given by

$$\mathbf{e}_{ij} = \begin{bmatrix} \Delta x_{ij} \\ \Delta y_{ij} \\ d_{ij} \end{bmatrix}, \tag{3.16}$$

where

$$\Delta x_{ij} = x_j - x_i, \qquad \Delta y_{ij} = y_j - y_i, \tag{3.17}$$

and

$$d_{ij} = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}. \tag{3.18}$$

These features provide the graph neural network with information about the relative position, orientation, and spatial separation of neighboring mesh nodes.

Table 3.1: Node features used as input to the graph neural network.

Feature	Dimension	Description
$u_i^n,\ v_i^n,\ p_i^n$	3	Flow state at node i
Δt_{n+1}	1	Time step between t_n and t_{n+1}
$\mathbf{g}_{i,\text{wall}}$	2	Wall-distance direction vector
$\mathbf{g}_{i,\text{inlet}}$	2	Inlet-distance direction vector
$\mathbf{g}_{i,\text{outlet}}$	2	Outlet-distance direction vector
$I_{i,\text{interior}}$	1	Interior-node indicator
$I_{i,\text{wall}}$	1	Wall-node indicator
$I_{i,\text{inlet}}$	1	Inlet-node indicator
$I_{i,\text{outlet}}$	1	Outlet-node indicator
Total	**14**	

3.4.1. Additional features

Apart from the previous features that have been used in the best configurations, during the development also other features have been considered as potentially helpful for the model performance but given the reduced improvement in performance and for the sake of simplicity of the model, they were not used in the more advanced phases of the project. In particular the following physical features have been experimented in an early stage:

- The four components of the velocity-gradient tensor:

$$\frac{\partial u}{\partial x}, \qquad \frac{\partial u}{\partial y}, \qquad \frac{\partial v}{\partial x}, \qquad \frac{\partial v}{\partial y}.$$

- The divergence of the convective acceleration:

$$\nabla \cdot \left[(\mathbf{u} \cdot \nabla)\mathbf{u} \right].$$

All these features where extracted directly from COMSOL Multiphysics [4] for each node.

3.5. Graph Dataset Construction

As previously discussed, predicting the temporal increment of the flow variables rather than the complete state at the next time step has been shown to be effective for mesh-based models [23]. Preliminary experiments have also shown this result. The network is therefore trained to predict

$$\Delta \mathbf{x}_{n+1} = \mathbf{x}_{n+1} - \mathbf{x}_n, \tag{3.19}$$

where the state at time t_n is defined as

$$\mathbf{x}_n = \begin{bmatrix} \mathbf{u}_n \\ \mathbf{p}_n \end{bmatrix}. \tag{3.20}$$

The graph dataset is constructed from the time-dependent COMSOL simulations according to the following procedure:

- A time-dependent simulation is performed for N_t time steps. At each time t_n, the velocity components u and v and the pressure p are stored at all FEM degrees of freedom.
- For each transition from t_n to t_{n+1}, a graph $\mathcal{G}_n$ is constructed using the flow state at t_n, together with the geometric, temporal, and boundary-related features described in the previous sections.
- The input to the GNN is the graph $\mathcal{G}_n$, while the supervised target consists of the nodal state increments

$$\Delta \mathbf{x}_{n+1} = \begin{bmatrix} \mathbf{u}_{n+1} - \mathbf{u}_n \\ \mathbf{p}_{n+1} - \mathbf{p}_n \end{bmatrix}. \tag{3.21}$$

Once the GNN has been trained, given a graph input $\mathcal{G}_n$ the predicted increment is added to the current state to obtain the prediction at the next time step,

$$\widehat{\mathbf{x}}_{n+1} = \mathbf{x}_n + \widehat{\Delta \mathbf{x}}_{n+1}. \tag{3.22}$$

Therefore, each pair of consecutive time-dependent snapshots generates one supervised graph sample corresponding to the transition $t_n \to t_{n+1}$.

3.6. Normalization

Normalization is a typical practice used in machine learning to improve efficiency and accuracy and also with graph neural networks it can help to improve results [2].

The physical quantities u, v, and p have been scaled and nondimensionalized as in [19]:

$$u^* = \frac{u}{U_{\text{mean}}}, \qquad v^* = \frac{v}{U_{\text{mean}}}, \qquad p^* = \frac{p}{\rho U_{\text{mean}}^2}. \tag{3.23}$$

where U_{mean} is the characteristic velocity. One important thing to notice is that u and v are scaled by the same quantity, hence there is no artificial drifting imposed by the network. Furthermore, it is important to notice that the different simulations were not scaled with the same quantity. Since the different simulations correspond to different Reynolds numbers, the characteristic velocity associated with each simulation is used for its nondimensionalization.

The timestep Δt has been scaled and nondimensionalized in the following way:

$$\Delta t^* = \frac{\Delta t U_{\text{mean}}}{L_{\text{ref}}}, \tag{3.24}$$

The characteristic length is defined as $L_{\text{ref}} = 2R$, where R denotes the cylinder radius for circular obstacles. For non-circular geometries, an equivalent radius based on the obstacle area A is introduced as $R_{\text{eq}} = \sqrt{A/\pi}$, such that $L_{\text{ref}} = 2R_{\text{eq}}$.

The geometry-related features are standardized independently using the mean and standard deviation computed exclusively over the training simulations. For each geometric feature g_j, the normalized value is therefore defined as

$$g_j^* = \frac{g_j - \mu_{g_j}}{\sigma_{g_j}},$$

where μ_{g_j} and σ_{g_j} denote the training-set mean and standard deviation of the corresponding feature. The binary indicators are not normalized.

4

Graph Neural Network architecture

Given the good results found in [12] with Graph Convolutional Neural Networks (GCNs) [13], a first attempt was to use this architecture for the prediction of [u,v,p] in each node. The results found initially suggested that this might not be the most suitable architecture for this problem. Because of this reason different architectures, especially designed for mesh-level predictions have been explored.

A first observation that has been done is that in the Navier Stokes equations velocity and pressure can vary and behave with different scales and in different manners, therefore a unified architecture used for both quantities is not desirable since it might be difficult to simultaneously learn these different behaviours. Furthermore, the possibility to have different architectures for these different quantities gives more flexibility in terms of implementation and is a useful tool to isolate better what might be the problem in the developing phase.

4.1. MeshGraphNet - velocity architecture

The architecture that has been used to predict velocity is MeshGraphNet [23] using NVIDIA PhysicsNeMo Framework [5] , which have been proved to provide good results in Mesh-Based Simulation [23]. The architecture is composed by three components: an encoder, one or multiple processors and a decoder.

The encoder role is to convert raw data into a unified, high-dimensional representation. Its role is to create rich embedding vectors for both the nodes and the edges of the graph where the GNN can more effectively learn patterns and relationships. The node encoder is a Multi-Layer Perceptron (MLP), takes the initial node features and maps them to a dense, abstract $d-$dimensional embedding vector. The edge encoder is a separate Multi-Layer Perceptron (MLP), that takes the initial edge features and produces a dedicated edge $d-$dimensional embedding [20]. The output of the encoders for node i and edge (i, j) will be:

$$\mathbf{v}_i^{(0)} \in \mathbb{R}^d, \qquad \mathbf{e}_{ij}^{(0)} \in \mathbb{R}^d. \tag{4.1}$$

The processor is the main part of the Graph Neural Network and it consists of three main parts: the edge update, the message aggregation and the node update. For the l-th processor the edge (i, j) update will be:

$$\mathbf{e}_{ij}^{(l+1)} = \mathbf{e}_{ij}^{(l)} + f_E^{(l)} \left(\mathbf{e}_{ij}^{(l)}, \mathbf{v}_i^{(l)}, \mathbf{v}_j^{(l)} \right). \tag{4.2}$$

Where $f_E^{(l)}$ will be a Multi-Layer Perceptron (MLP). The message aggregation phase for the l-th processor at node i will be some kind of aggregation of its adjacent edges information (for example in our case it is the sum, but it can also be the mean or other aggregation functions):

$$\mathbf{m}_i^{(l+1)} = \sum_{j \in \mathcal{N}(i)} \mathbf{e}_{ij}^{(l+1)}. \tag{4.3}$$

For the l-th processor the node i update will be:

$$\mathbf{v}_i^{(l+1)} = \mathbf{v}_i^{(l)} + f_V^{(l)} \left(\mathbf{v}_i^{(l)}, \mathbf{m}_i^{(l+1)} \right). \tag{4.4}$$

Where $f_V^{(l)}$ is a Multi-Layer Perceptron (MLP).

Clearly, as the number of processors increases the graph will communicate with elements on a longer distance, but increasing too much the number of processors can reflect in an over-smoothing effect [25], preventing the recognition of velocity peaks that might occur in some mesh areas during the simulations.

The decoder is the final component, translating the high-level embeddings from the processor back into the original dimensional prediction. A final MLP takes the updated node embeddings and outputs the predicted change for each node. [20].

4.2. Bi-stride Multi-Scale GNN - Pressure Architecture

The mathematical reason that motivates this choice of architecture for the pressure prediction is the following. We start by applying the divergence operator on both sides of the momentum equation:

$$\nabla \cdot (\rho \left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} \right)) = \nabla \cdot (-\nabla p + \mu \nabla^2 \mathbf{u}), \tag{4.5}$$

Assuming constant density and viscosity, the divergence operator can be distributed over the different terms:

$$\rho \nabla \cdot \left(\frac{\partial \mathbf{u}}{\partial t} \right) + \rho \nabla \cdot ((\mathbf{u} \cdot \nabla)\mathbf{u}) = -\nabla \cdot (\nabla p) + \mu \nabla \cdot \left(\nabla^2 \mathbf{u} \right). \tag{4.6}$$

For an incompressible flow, the continuity equation requires:

$$\nabla \cdot \mathbf{u} = 0. \tag{4.7}$$

Therefore, assuming a sufficient regularity of $\mathbf{u}$, the divergence of the temporal term vanishes:

$$\nabla \cdot \left(\frac{\partial \mathbf{u}}{\partial t} \right) = \frac{\partial}{\partial t} (\nabla \cdot \mathbf{u}) = 0. \tag{4.8}$$

Similarly, for constant viscosity:

$$\nabla \cdot \left(\nabla^2 \mathbf{u} \right) = \nabla^2 (\nabla \cdot \mathbf{u}) = 0. \tag{4.9}$$

Moreover,

$$\nabla \cdot (\nabla p) = \nabla^2 p. \tag{4.10}$$

The divergence of the momentum equation, therefore reduces to

$$\boxed{\nabla^2 p = -\rho \nabla \cdot [(\mathbf{u} \cdot \nabla)\mathbf{u}]}. \tag{4.11}$$

Equation (4.11) shows that, given the assumptions of constant density and viscosity and sufficient regularity of the velocity, for incompressible flow, the pressure field is governed by a Poisson equation [10]. This is an elliptic equation, meaning that the pressure at a given location is not determined only by the local state of the flow but is influenced by the velocity field and boundary conditions throughout

the domain [8]. Consequently, pressure follows a behaviour that depends on global information from the whole domain.

In order to provide to the nodes global information of the graph, it would not be feasible computationally to increase severely the depth of the MeshGraphNet [3] and furthermore possibly we could encounter the phenomenon of oversmoothing [25]. Because of these reasons, for the pressure predictions a Bi-stride Multi-Scale GNN [3] has been used, using NVIDIA PhysicsNeMo Framework [5] .

This is a multi-scale variant that extends the base MeshGraphNet model with a U-Net-style architecture [3]. It creates a graph pyramid consisting of multiple levels of the graph at different resolutions. The model applies a bi-stride message-passing component over these levels to capture both local, fine-grained features and global, coarse-grained features more effectively. At each coarsening step, the bi-stride procedure selects a subset of nodes from the current graph and constructs a lower-resolution graph, without the need for the manual drawing of coarser meshes and avoiding the wrong edges by spatial proximity [3], allowing information to travel over increasingly larger spatial distances [20]. Qualitatively, this might be considered similar to the use of multiple resolution levels in a multigrid numerical solver.

What is done in practice after the initial encoding is to first create multiple graphs

$$\mathcal{G}_l = (\mathcal{V}_l, \mathcal{E}_l), \qquad l = 1, \ldots, L, \tag{4.12}$$

where $l = 1$ represents the finest level, corresponding to the original input graph. To construct the coarser levels, the intuition is that, given the K-th order adjacency matrix of a graph $\mathbf{A}^K$, if

$$\left(\mathbf{A}^K\right)_{ij} > 0, \tag{4.13}$$

then node j can be reached from node i through a path of K hops. When interpreted as a Boolean connectivity matrix, this information can therefore be used to construct connections between nodes that are not directly connected in the original graph.

The Bi-Stride pooling strategy exploits this property to construct progressively coarser graphs while preserving the original mesh topology. At each level l, a seed node s_l is first selected and a Breadth-First Search (BFS) is performed on the graph. Each node is therefore associated with its geodesic distance from the seed,

$$d_i = \mathrm{dist}_{\mathcal{G}_l}(s_l, i). \tag{4.14}$$

Bi-Stride pooling selects a subset of the nodes. For example, one possible selection is

$$\mathcal{I}_l = \{ i \in \mathcal{V}_l \ : \ d_i \bmod 2 = 0 \}, \tag{4.15}$$

where $\mathcal{I}_l$ denotes the set of nodes retained at the next coarser level. The corresponding node set is therefore

$$\mathcal{V}_{l+1} = \mathcal{I}_l \subset \mathcal{V}_l. \tag{4.16}$$

Since the selected nodes generally do not remain directly connected after the intermediate nodes are removed, the adjacency matrix of the next level is then obtained by keeping only the rows and columns associated with the selected nodes:

$$\boxed{\mathbf{A}_{l+1} = \mathbf{A}_l^2[\mathcal{I}_l, \mathcal{I}_l]}. \tag{4.17}$$

This procedure is recursively repeated to construct the complete hierarchy

$$\mathcal{G}_1 \to \mathcal{G}_2 \to \cdots \to \mathcal{G}_L, \tag{4.18}$$

where the number of nodes progressively decreases, increasing the receptive field of message propagation. Since the construction relies on graph connectivity rather than Euclidean proximity, it preserves the topology of the original mesh and avoids introducing artificial connections between nodes that may be spatially close but separated by a physical boundary [3]. The details of how the weights of the nodes features are handled in the downsampling and upsampling can be found in [3]. The figure 4.1 shows a diagram of the architecture.

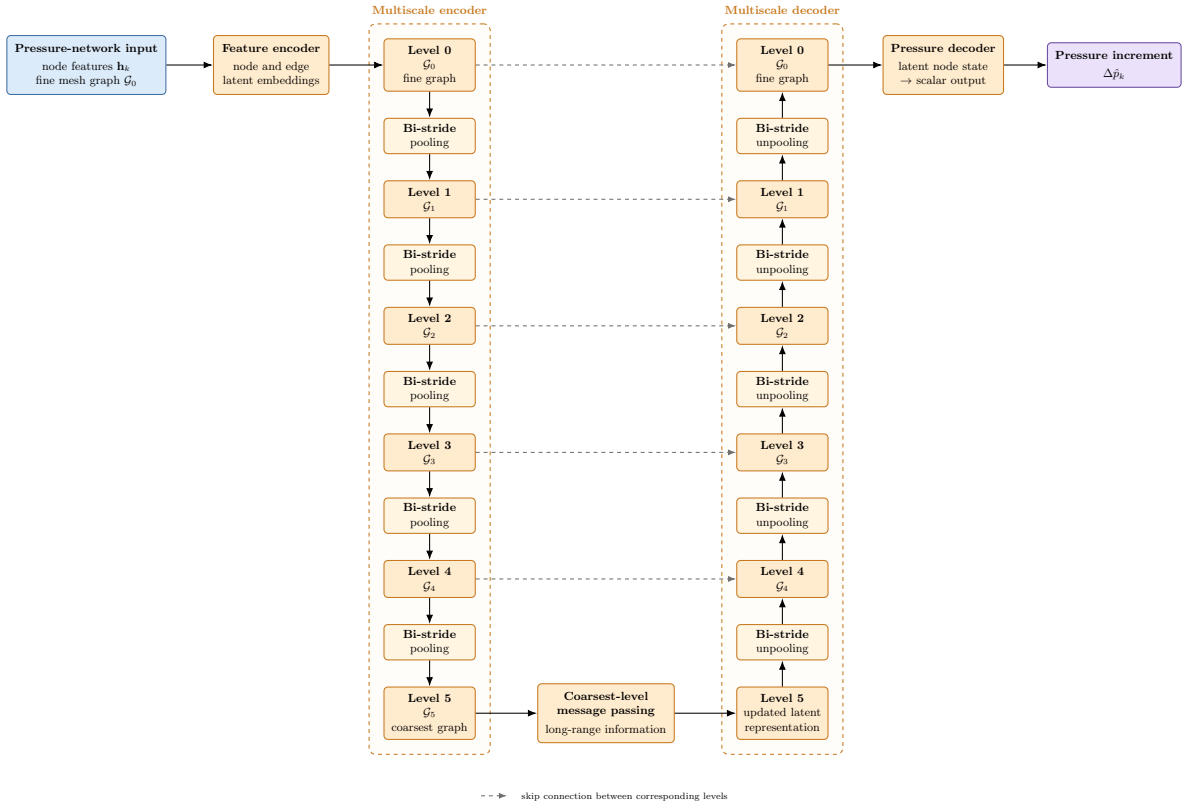


Figure 4.1: Architecture of the Bi-Stride Multi-Scale (BSMS) graph neural network used for pressure prediction.

4.3. Initialization of the network parameters

The initialization of the network parameters can influence the results in terms of performance. For example, classic initialization methods such as *Xavier initialization* seek to avoid vanishing gradients and maintain steady information flow [16].

In our case, we noticed that for time-dependent simulations that tend to converge to a stationary solution, the solution of the previous time step can sometimes be already sufficiently close to the solution for the next time step, so that the solver needs just 4 or 5 iterations to converge. Because of this reason, with a standard initialization, the GNN prediction could perform worse than with the initial guess of the solution of the previous timestep. To mitigate this phenomenon, the last linear layer of the GNN is initialized with all zero weights. Since we are predicting the update $\Delta u, \Delta v, \Delta p$, with a zero initialization, assuming that our loss function is monotonically non increasing, our prediction should at least be not worse than the standard initial guess in terms of the function that we are minimizing in our loss.

Another option to obtain the same result would be to set to zero initially all the weights of the network. However this could possibly give unstable result, losing the advantages of the standard initialization provided by NVIDIA implementation[20].

4.4. Loss functions

As explained in Section 2.7 the correspondence between the minimization of our loss functions and its relationship with the Newton residual is non-trivial. For this reason different loss functions have been explored. The implementation allows to keep this loss terms together in a comprehensive loss function assigning a weight to each contribution which can be fixed or also dynamic, which can have some advantages as shown in [17].

4.4.1. Physics-based residual loss

The first approach considered was to explicitly incorporate the physics of the Navier–Stokes equations into the training objective. In particular, the incompressible Navier–Stokes equations are given by

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u}\right) = -\nabla p + \mu\nabla^2\mathbf{u} + \mathbf{f}, \tag{4.19}$$

together with the incompressibility constraint

$$\nabla\cdot\mathbf{u} = 0. \tag{4.20}$$

These equations can equivalently be written in residual form. The momentum residual is defined as

$$\mathbf{R}_{\text{mom}} = \rho\left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u}\right) + \nabla p - \mu\nabla^2\mathbf{u} - \mathbf{f}, \tag{4.21}$$

such that

$$\mathbf{R}_{\text{mom}} = \mathbf{0}, \tag{4.22}$$

while the continuity residual is defined as

$$\mathbf{R}_{\text{cont}} = \nabla\cdot\mathbf{u}, \tag{4.23}$$

such that

$$\mathbf{R}_{\text{cont}} = 0. \tag{4.24}$$

The momentum residual consists of two components for each mesh cell, corresponding to the two spatial components of the velocity field $\mathbf{u}$. To obtain a scalar momentum loss for each node, we consider the average of the squared momentum residuals for each node. Since the computational mesh is unstructured and does not correspond to a regular Cartesian grid, standard finite-difference schemes cannot be directly used to estimate the required spatial derivatives. Instead, the derivatives were approximated using the Weighted Least-Squares (WLSQ) method adopted in [18].

This approach, however, presents some limitations. Firstly, the derivatives obtained through the WLSQ approximation may not be sufficiently accurate. For instance, the time derivative is approximated using BDF1 (as show in Formula 2.2) because otherwise, the whole architecture would change, needing as inputs not just the values of the nodes of the current solution but also of the previous one. Secondly, the reference FEM solution is the discrete solution of the weak form of the governing equations. Therefore, there is no theoretical guarantee that the strong-form residuals computed through the WLSQ approximation are exactly zero when evaluated on the FEM ground-truth solution.

The physics residuals evaluated on the GNN prediction were normalized using reference values computed from the ground-truth FEM solutions in the training set. The residuals are first evaluated over the mesh cells and reduced to a mean squared residual. The resulting scalar loss is then divided by the corresponding reference value obtained by averaging the same quantity over all training transitions.

Denoting the GNN prediction by $\widehat{\mathbf{x}}$, the normalized continuity loss is defined as

$$\widetilde{\mathcal{L}}_{\text{cont}} = \frac{\mathcal{L}_{\text{cont}}(\widehat{\mathbf{x}})}{\mathcal{L}_{\text{cont}}^{\text{ref}}}, \qquad \mathcal{L}_{\text{cont}}^{\text{ref}} = \frac{1}{N_{\text{train}}} \sum_{k=1}^{N_{\text{train}}} \mathcal{L}_{\text{cont}} \left(\mathbf{x}_k^{\text{FEM}}\right), \tag{4.25}$$

and analogously for the momentum residual.

The normalized physics terms are then introduced as regularization terms in the overall training objective,

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda_{\text{cont}} \widetilde{\mathcal{L}}_{\text{cont}} + \lambda_{\text{mom}} \widetilde{\mathcal{L}}_{\text{mom}}. \tag{4.26}$$

where λ_{cont} and λ_{mom} control the relative contribution of the continuity and momentum residuals, respectively. The introduction of these terms in the loss function did not help in the reduction of Newton iterations, however it was observed that when the GNN was actually reducing the number of Newton iterations, $\mathcal{L}_{\text{cont}}$ and $\mathcal{L}_{\text{mom}}$ were closed to 1, so this metric could possibly be seen as a physical sanity check.

Note that the current implementation assumes that our mesh is composed of triangular elements, this is particular important since to compute the residuals, volume expressions of the derivatives are used, therefore these physical terms would not be applicable to different kind of meshes.

4.4.2. Jacobian scaled loss

Let us consider the Newton residual $\mathbf{R}(\mathbf{u})$ and the ground-truth solution $\mathbf{u}^{n+1}$. Assuming that the GNN prediction $\widehat{\mathbf{u}}$ is sufficiently close to the ground-truth solution, the residual can be approximated using a first-order Taylor expansion around $\mathbf{u}^{n+1}$:

$$\mathbf{R}_{n+1}(\widehat{\mathbf{x}}) \approx \mathbf{R}_{n+1}(\mathbf{x}^{n+1}) + \mathbf{J}_{n+1}(\mathbf{x}^{n+1}) \left(\widehat{\mathbf{x}} - \mathbf{x}^{n+1}\right), \tag{4.27}$$

where

$$\mathbf{J}_{n+1}(\mathbf{x}^{n+1}) = \left.\frac{\partial \mathbf{R}_{n+1}(\mathbf{x})}{\partial \mathbf{x}}\right|_{\mathbf{x}^{n+1}} \tag{4.28}$$

is the Jacobian matrix of the nonlinear FEM residual evaluated at the ground-truth solution.

Since $\mathbf{x}^{n+1}$ is the converged solution of the nonlinear system,

$$\mathbf{R}_{n+1}(\mathbf{x}^{n+1}) \approx \mathbf{0}, \tag{4.29}$$

and therefore the residual associated with the GNN prediction can be approximated as

$$\mathbf{R}_{n+1}(\widehat{\mathbf{x}}) \approx \mathbf{J}_{n+1}(\mathbf{x}^{n+1}) \left(\widehat{\mathbf{x}} - \mathbf{x}^{n+1}\right). \tag{4.30}$$

Since the objective is to obtain a GNN prediction characterized by a small Newton residual, the Jacobian-weighted prediction error can therefore be considered in the loss function. The Jacobian-based loss is defined as

$$\mathcal{L}_J = \frac{1}{N_{\text{dof}}} \left\| \mathbf{J}_{n+1}(\mathbf{x}^{n+1}) \left(\widehat{\mathbf{x}} - \mathbf{x}^{n+1}\right) \right\|_2^2. \tag{4.31}$$

In contrast to the standard MSE, which assigns the same importance to the prediction error independently of its effect on the nonlinear FEM system, this formulation weights the prediction error through the Jacobian of the residual evaluated at the ground-truth solution.

Since the Jacobian is assembled in physical FEM variables, the prediction error used in this loss must also be expressed in physical units. Therefore, the GNN prediction is firstly rescaled and brought to the respective physical dimensions before computing

$$\mathbf{e}_{\text{phys}} = \widehat{\mathbf{x}}_{\text{phys}} - \mathbf{x}_{\text{phys}}^{n+1}, \tag{4.32}$$

and the corresponding loss becomes

$$\mathcal{L}_J = \frac{1}{N_{\text{dof}}} \left\| \mathbf{J}_{n+1}(\mathbf{x}_{\text{phys}}^{n+1}) \mathbf{e}_{\text{phys}} \right\|_2^2 \tag{4.33}$$

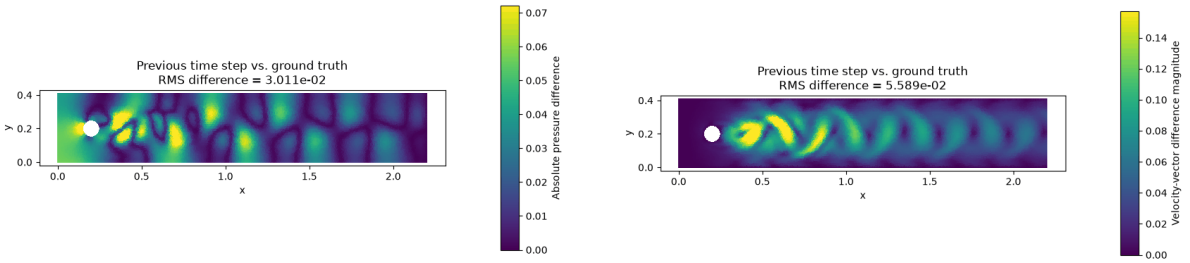
In practice, in order to have loss terms on the same scale, the Jacobian-based loss is normalized with respect to the linearized residual obtained when using the previous time-step solution $\mathbf{u}^n$ as the initial guess. This provides a measure of the quality of the GNN prediction relative to the standard initial guess used by the time-stepping procedure. The normalized Jacobian loss is therefore defined as

$$\boxed{\mathcal{L}_{J,\text{norm}} = \frac{\left\| \mathbf{J}_{n+1} \left(\widehat{\mathbf{x}}_{\text{phys}} - \mathbf{x}_{\text{phys}}^{n+1} \right) \right\|_2^2}{\left\| \mathbf{J}_{n+1} \left(\mathbf{x}_{\text{phys}}^{n} - \mathbf{x}_{\text{phys}}^{n+1} \right) \right\|_2^2 + \varepsilon}} \tag{4.34}$$

where ε is a small numerical constant introduced to avoid division by zero. Consequently, values smaller than one indicate that the GNN prediction produces a smaller linearized nonlinear residual than the previous time-step solution, while values above one indicate a worse initial guess according to this metric.

4.4.3. Top-k loss

After some initial experiments, it has been noticed that in a number of situation the error between the real solution and the previous timestep is not spread equally around the mesh but is localized in some specific areas (where the actual fluid transient is happening). As an example, in Figure 4.2 there is a visualization of the error distribution between the previous timestep solution and the target timestep solution. Based on this observation, a loss function that could penalize more specifically the points



(a) Pressure error distribution.

(b) Velocity error distribution.

Figure 4.2: Spatial distribution of the error for the standard initial guess at $t = 7.0\,\text{s}$ for the $\text{Re} = 100$ configuration. The pressure error is shown on the left and the velocity-vector error magnitude on the right.

(hence, the nodes) that have the largest loss was implemented. In particular what was implemented is the Average Top-k Loss which has the great advantage of remaining a convex function over all individual losses, which can lead to convex optimization problems that can be solved effectively with conventional gradient-based methods [6].

4.5. Hard enforcement boundary condition

Based on PINNs (Physics Informed Neural Networks) [24] idea of enforcing boundary conditions in a Neural Network and based on the good results obtained in [18] by hardly enforcing boundary conditions

for the Navier-Stokes equation in GNNs, it has been implemented a hard enforcement of boundary conditions. In particular, the velocity has been hard-enforced on the inlet boundary and on the wall boundaries and the pressure has been hard-enforced on the outlet boundary according to the values that were set in the COMSOL Multiphysics model.

The enforcement was implemented both during the training and in the evaluation. During the training, once the GNN had computed the forward pass, before computing the loss the boundary condition values were imposed on the nodes. During the implementation, once the GNN had computed its forward pass with the trained architecture, the boundary nodes values were substituted with the boundary conditions.

5

Results

5.1. Results on Reynolds numbers dataset

The results presented in this section were obtained using the hyperparameter configuration reported in Table B.1. The corresponding solver performance across the different Reynolds numbers is summarized in Table 5.1.

Table 5.1: Comparison of the total number of nonlinear iterations required by the stationary solver using the standard and GNN-predicted initial guesses. Positive values of the iteration reduction indicate an improvement obtained with the GNN initial guess. Each dataset contains 349 transitions.

Re	Standard	GNN	Iterations saved	Reduction (%)
		Train Set		
40	1336	1462	−126	−9.43
80	2335	2037	298	12.76
120	3024	2449	575	19.01
160	3488	2986	502	14.39
180	3699	3224	475	12.84
		Test Set		
60	1884	1779	105	5.57
100	2756	2274	482	17.49
140	3228	2745	483	14.96
200	3841	3434	407	10.60

The first thing that might be noticed is that the GNN initialization is certainly not helping the configuration with Re=40. The reason for this behavior is that at 40 Re the flow is converging almost to a stationary solution, hence the dynamics represented in its simulation might be under-represented on the train set. However, it should also be noticed that the number of iterations with the Standard initialization has an average of approximately 3,82 iterations per timestep for the 40 Re simulation, therefore it is inherently a difficult task to improve this performance compared to other simulations and might be considered also outside of the main scope of this project.

Nevertheless, the results demonstrate a reduction in Newton iterations across several Reynolds-number configurations, with the largest improvement in the test set observed at Re = 100, where a reduction of 17.49% was achieved. Furthermore, the positive results obtained for Reynolds numbers not included in the training dataset suggest promising generalization capabilities of the proposed approach.

It has also been tested how well the model is able to help the solver using COMSOL's adaptive timestep configuration [4] which is arguably what may be used in common practice by the user. The results are shown in Table 5.2.

Table 5.2: Comparison of the total number of nonlinear iterations required by the stationary solver using the standard and GNN-predicted initial guesses with adaptive time step. Positive values of the iteration reduction indicate an improvement obtained with the GNN initial guess.

Re	Standard	GNN	Iterations saved	Reduction (%)
			Test Set	
40	464	506	−42	−9.05
60	1488	1397	91	6.11
80	3389	3169	220	6.49
100	5331	4803	528	9.90
120	7824	7269	555	7.09
140	10165	9284	881	8.67
160	12246	11388	858	7.01
180	15322	14196	1126	7.35
200	17905	16852	1053	5.88

The results obtained with adaptive time stepping show a reduction in the effectiveness of the GNN-based initialization compared to the fixed time-step experiments. The GNN still provides a reduction in Newton iterations for almost all configurations, although the improvements are generally smaller.

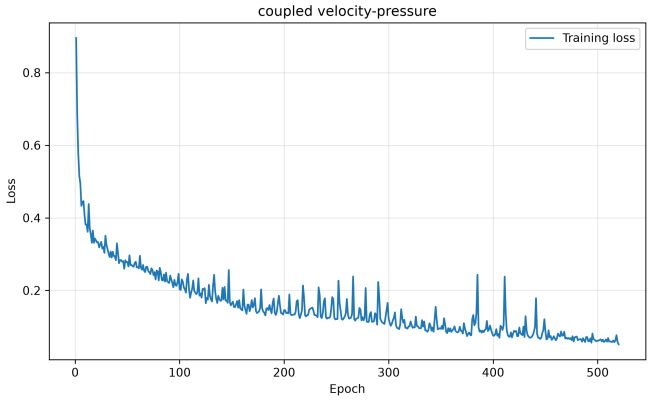


Figure 5.1: Training loss.

5.1.1. Jacobian scaled loss results

For the Jacobian scaled loss model, the hyperparameter configuration that has been used is shown in Table B.3. The hyperparameters are the same as the data loss configuration, which changes is simply the type of loss function, but the penalty for the top-k loss and the penalized percentage are the same for a fair comparison. The results on the different simulations are the following:

Table 5.3: Comparison of the total number of nonlinear iterations required by the stationary solver using the standard and GNN-predicted initial guesses. Positive values of the iteration reduction indicate an improvement obtained with the GNN initial guess. Each dataset contains 349 transitions.

Re	Standard	GNN	Iterations saved	Reduction (%)
		Train Set		
40	1336	1484	−148	−11.08
80	2335	2283	52	2.23
120	3024	2805	219	7.24
160	3488	3332	156	4.47
180	3699	3493	206	5.56
		Test Set		
60	1884	1888	−4	−0.21
100	2756	2589	167	6.06
140	3228	3116	112	3.47
200	3841	3628	213	5.55

The results show a similar pattern to those shown in Table 5.1, however, the improvements are more limited, and with low Reynolds numbers the results exhibit a similar poor performance. Therefore, the data loss despite the mathematical limitations described in Section 2.7, achieves better results. The hypothesis of closeness to the ground truth solution might not be satisfied in all the time steps, and this could be a motivation for the poor performance results obtained.

5.2. Results on different geometries dataset

The first result shown has been obtained with the hyperparameters configuration shown in Table B.1 (the same as the first Reynolds configuration) but with batch size 1 for computational feasibility. The results of the different simulations are the following:

Table 5.4: Comparison of the total number of nonlinear iterations required by the stationary solver using the standard and GNN-predicted initial guesses. Positive values of the iteration reduction indicate an improvement obtained with the GNN initial guess. Each dataset contains 349 transitions.

Configuration	Standard	GNN	Iterations saved	Reduction (%)
		Train Set		
Cfg1	2156	2011	145	6.73
Cfg2	2898	2171	727	25.09
Cfg3	2711	2441	270	9.96
Cfg4	2074	1761	313	15.09
Cfg5	2413	2311	102	4.23
		Test Set		
Tri quad	3531	3258	273	7.73
Cylinder stretch	2942	3244	−302	−10.27
Mixed all	3420	3224	196	5.73

A first observation is that the GNN prediction generalizes poorly to the cylinder stretch configuration, resulting in an increase in the number of Newton iterations required during the simulation. One possible explanation is that the obstacle is larger than those encountered during the training phase. However, the main reason for the poor performance may be related to the flow behavior in this configuration. During the initial part of the simulation, the flow exhibits vortex dynamics similar to those associated with the Kármán vortex street observed for the cylindrical training configurations. In the later part of the simulation, however, the flow gradually converges towards a stationary solution, representing a flow regime that was essentially never encountered during training. This can also be seen in Figure 5.2.

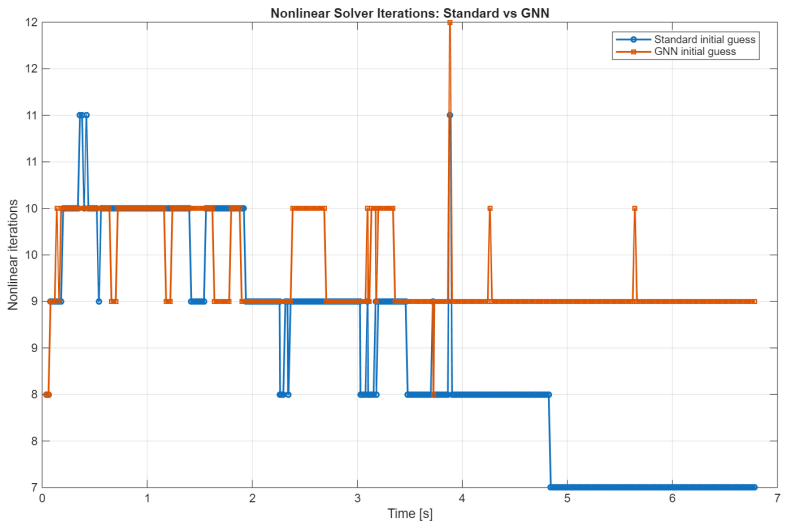


Figure 5.2: Number of Newton iterations for the cylinder stretch configuration.

It has also been tested, as in the Reynolds numbers dataset, how well the model is able to help the solver using COMSOL's adaptive timestep configuration [4] which is arguably what may be used in common practice by the user. The results are shown in Table 5.5.

Table 5.5: Comparison of the total number of nonlinear iterations required by the stationary solver using the standard and GNN-predicted initial guesses with adaptive time step. Positive values of the iteration reduction indicate an improvement obtained with the GNN initial guess.

Configuration	Standard	GNN	Iterations saved	Reduction (%)
		Test Set		
Cfg1	3537	3624	−87	−0.76
Cfg2	5747	5549	198	3.44
Cfg3	5631	5662	−31	−0.55
Cfg4	3870	3917	−47	−1.21
Cfg5	4520	4534	−14	−0.31
Tri quad	7673	7565	108	1.41
Cylinder stretch	3732	3994	−262	−7.02
Mixed all	7440	7593	−123	−1.65

In the majority of these experiments, the GNN initialization leads to an increase in the number of Newton iterations for several configurations. One possible explanation is related to the choices made in defining the time-step feature Δt, which may not be sufficiently robust when applied to configurations outside the training distribution. For instance, the choice of using an equivalent diameter as the characteristic length for non-circular obstacles is somewhat arbitrary and may influence the resulting predictions and, consequently, the solver performance.

5.2.1. Jacobian scaled loss results

For the Jacobian scaled loss model, the hyperparameter configuration that has been used is shown in Table B.3. The hyperparameters are the same of the data loss configuration, which changes is simply the type of loss function, but the penalty for the top-k loss and the penalized percentage are the same for a fair comparison. The results on the different simulations are the following:

Table 5.6: Comparison of the total number of nonlinear iterations required by the stationary solver using the standard and GNN-predicted initial guesses. Positive values of the iteration reduction indicate an improvement obtained with the GNN initial guess. Each dataset contains 349 transitions.

Configuration	Standard	GNN	Iterations saved	Reduction (%)
Train Set				
Cfg1	2156	2110	46	2.13
Cfg2	2898	2504	394	13.60
Cfg3	2711	2437	274	10.11
Cfg4	2074	1865	209	10.08
Cfg5	2413	2238	175	7.25
Test Set				
Tri quad	3531	3315	216	6.12
Cylinder stretch	2942	3268	−326	−11.08
Mixed all	3420	3309	111	3.25

Similarly to the results obtained with the Jacobian-scaled loss on the Reynolds-number dataset, the model exhibits trends comparable to those observed with the data-based loss, although with generally poorer performance in terms of Newton iteration reduction. Furthermore, the increase of Newton iterations for the cylinder-stretch configuration suggests that the model's generalization capabilities may be also limited by the diversity of flow regimes represented in the training dataset.

5.3. Hyperparameter optimization results

Due to the high computational cost associated with GNN training, a grid search over the hyperparameter space was computationally infeasible. Following preliminary experiments aimed at assessing the sensitivity of the model to different hyperparameters, an optimization procedure was conducted. The search focused on three hyperparameters of the velocity network: the learning rate, the number of hidden neurons, and the number of message-passing steps in the processor.

Hyperparameter optimization was performed using Optuna framework [1]. The objective was to minimize the validation loss by exploring different combinations of the selected hyperparameters. The optimization procedure also incorporated an early-stopping mechanism for unpromising trials, allowing computational resources to be allocated more efficiently. Additionally, a maximum training duration of 96 hours was imposed for each trial. The investigated hyperparameter ranges and the corresponding optimization settings are reported in Table B.2.

The validation loss histories for the five investigated configurations are shown in Figure 5.3. The curves exhibit a broadly similar trend, with most of the improvement occurring during approximately the first 100 epochs, suggesting a relative small sensitivity to the changing hyperparameters considered. Beyond this point, the validation loss tends to increase, suggesting an overfitting trend [26]. This phenomenon can also be confirmed by Figure 5.5 which shows the Training and Validation loss of the best configuration over the epochs.

The relative importance of the investigated hyperparameters, as estimated by Optuna, is illustrated in Figure 5.4. The hyperparameter configuration achieving the lowest validation loss is reported below.

$$\begin{aligned} \text{Velocity learning rate} &= 6.552308 \times 10^{-5} \\ \text{Velocity number of neurons} &= 83 \\ \text{Velocity processor size} &= 10 \end{aligned}$$

The results in terms of Newton iterations with the best hyperparameter configuration found are shown in Table 5.7.

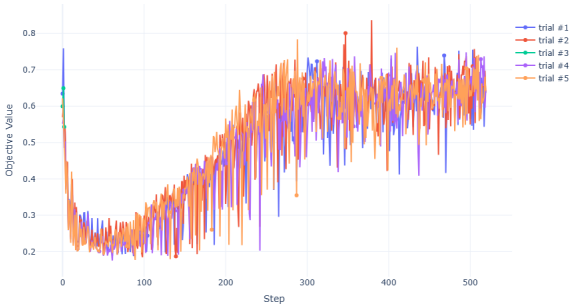


Figure 5.3: Validation losses during training.

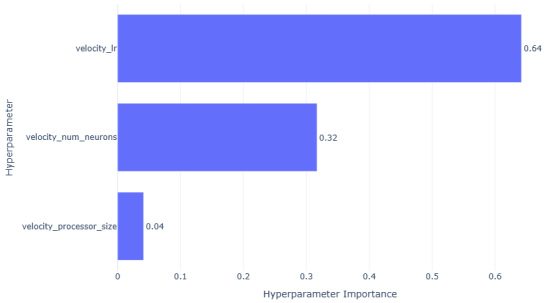


Figure 5.4: Hyperparameter relative importance.

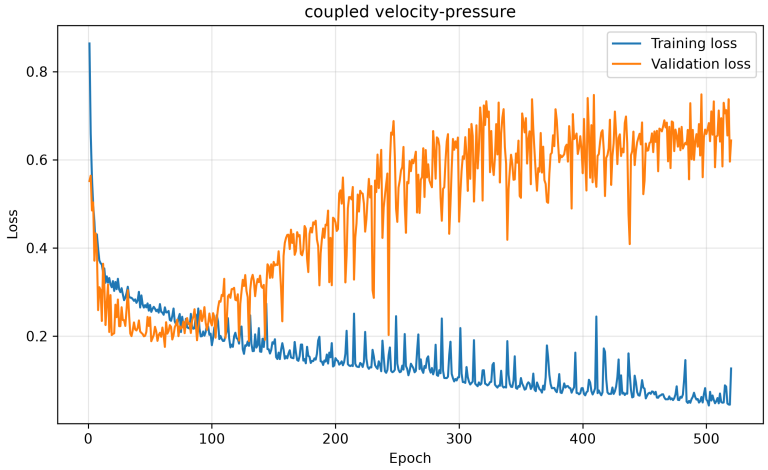


Figure 5.5: Training and Validation loss for the best configuration.

Table 5.7: Comparison of the total number of nonlinear iterations required by the stationary solver using the standard and GNN-predicted initial guesses. Positive values of the iteration reduction indicate an improvement obtained with the GNN initial guess. Each dataset contains 349 transitions.

Configuration	Standard	GNN	Iterations saved	Reduction (%)
		Train Set		
Re 40	1336	1747	−411	−30.76
Re 80	2335	2282	53	2.27
Re 120	3024	2441	583	19.28
Re 160	3488	3208	280	8.03
Re 180	3699	3419	280	7.57
		Validation Set		
Re 60	1889	1841	48	2.54
Re 100	2756	2423	333	12.08
Re 140	3228	2894	334	10.35
		Test Set		
Re 200	3841	3604	237	6.17
Tri quad	3531	3305	226	6.40
Cylinder stretch	2942	3236	−294	−9.99
Mixed all	3420	3368	52	1.52

Surprisingly, the results obtained after hyperparameter optimization show worse performance in terms of Newton iterations compared to those obtained with the initial configuration reported in Table **??** for the

different Reynolds number datasets. A possible explanation for this unexpected result is, once again, the mismatch between the objective function minimized during hyperparameter optimization, which is based on the data loss between the predicted and ground-truth solutions, and the actual metric we aim to minimize: the number of Newton iterations required for convergence.

One possible hypothesis is that the initial, simpler model, despite exhibiting signs of overfitting in terms of data loss after approximately 100 epochs, may have indirectly learned to reduce the residual of the algebraic equations presented in Equation 2.6. A more targeted analysis would therefore be required to establish whether such a correlation exists and to what extent it influences the solver performance.

6

Conclusion

In this project we have implemented a machine learning framework to reduce the number of Newton iterations in the time dependent Navier Stokes equation in COMSOL Multiphysics. In particular, a GNN architecture has been designed to predict the velocity field $\mathbf{u}$ and the pressure field p at each new time step to provide a better initial guess to the nonlinear solver, the prediction was then used in COMSOL Multiphysics and compared with the standard initialization to evaluate the reduction of Newton iterations in an adapted stationary solver. The velocity network is based on a MeshGraphNet architecture while the pressure network is based on a BSMS architecture.

The numerical results demonstrate that the proposed GNN-based initialization can reduce the number of Newton iterations required by the nonlinear solver across a range of flow configurations. In the Reynolds-number generalization experiment, reductions of 5.57%, 17.49%,14.96%, and 10.60% were obtained for the unseen Reynolds numbers (Re=60), (100), (140), and (200), respectively. These results indicate that the model can generalize to Reynolds numbers not included in the training dataset, both through interpolation within the training range and limited extrapolation beyond it. This suggests that the network captures transferable features of the flow evolution rather than simply reproducing individual training trajectories.

The results also suggest that the effectiveness of the GNN initialization depends on the flow regimes seen during training . In particular, the model performed worse than the standard initialization for the (Re=40) training configuration. This behavior may be attributed to two factors: the limited representation of this flow regime in the training dataset and the fact that, as the flow approaches a stationary state, the previous time-step solution already provides a highly effective initial guess. Consequently, there is relatively little room for improvement, while prediction errors introduced by the GNN can instead increase the number of nonlinear iterations.

Similarly, the results on unseen geometries show a reduction of (7.73%) for the cylinder-tri-quad configuration and of (5.73%) for the mixed-obstacle configuration. In contrast, the cylinder-stretch configuration resulted in an (10.27%) increase in Newton iterations. In this case, the flow progressively reaches a nearly stationary regime during the later part of the simulation, which differs substantially from the predominantly unsteady dynamics represented in the training data, showing potentially a similar behavior to the (Re=40) configuration. These results suggest that the generalization capabilities of the model depend importantly on whether the dynamical regime encountered at prediction time is sufficiently represented during training.

The jacobian-scaled loss models trained and tested on different Reynolds numbers and on different geometries show weaker performances on the train and on the test set. Nevertheless, it is important to emphasize that the proposed loss relies on a Jacobian-based approximation of the residual, derived from a first-order Taylor expansion, rather than on the residual equations themselves. Consequently, when the prediction lies outside the region where the linear approximation is sufficiently accurate, minimizing the Jacobian-scaled loss may not effectively reduce the actual nonlinear residual.

Based on the good results for the data-loss models, a hyperparameter optimization was conducted to investigate if better performance could be achieved. The results show that, despite the initial model being overfitting, it still performed better in terms of Newton iterations for the majority of train and test samples compared to the best model obtained in the hyperparameter tuning procedure, this suggests that the relationship between prediction accuracy and nonlinear solver performance is more complex than a direct correlation and motivates the development of solver-aware training objectives.

Overall, these experiments show that improving the initial guess of a nonlinear solver through a learned model is feasible and can lead to meaningful reductions in nonlinear iterations. At the same time, the results confirm that prediction accuracy alone is not sufficient to guarantee improved Newton convergence. The quality of a learned initial guess must ultimately be assessed in terms of its interaction with the nonlinear solver, and its effectiveness can vary depending on both the geometry and the underlying flow regime.

While the results of this work demonstrate the potential of GNN-based initialization for reducing the computational effort of nonlinear solvers, several limitations remain and motivate further investigation. Firstly, the losses that have been used take into account just partially the goal of reducing the number of Newton iterations, hence what might be done in the future is to implement FEM residual functions in the Machine Learning architecture, ensuring its differentiability and using it as a loss function. Secondly, while conventional GNN architectures operate on discrete graph representations of the computational domain, Graph Neural Operators [15] provide a framework for learning mappings between infinite dimensional function spaces. Investigating these architectures could be particularly interesting for improving generalization across different mesh resolutions and discretizations, potentially extending the applicability of the proposed approach. Finally, the initial transient regime was excluded from the training data in the present work because of its substantially different dynamics. A dedicated investigation of this regime could be particularly interesting.

In conclusion, this work shows the potential of combining novel machine learning methods with classical mathematical methods for improving performances of CFD simulations in the laminar flow regime. The GNN is used only to provide an improved initial guess, while the final solution is still obtained by a FEM formulation and a Newton solver in COMSOL Multiphysics. The results obtained in this work therefore provide a promising basis for further investigation of data-driven warm-start strategies and, more generally, of hybrid approaches in which machine learning and numerical solvers complement each other.

References

[1] Takuya Akiba et al. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019.

[2] Tianle Cai et al. *GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training*. 2021. arXiv: 2009.03294 [cs.LG]. URL: https://arxiv.org/abs/2009.03294.

[3] Yadi Cao et al. *Efficient Learning of Mesh-Based Physical Simulation with BSMS-GNN*. 2026. arXiv: 2210.02573 [cs.LG]. URL: https://arxiv.org/abs/2210.02573.

[4] COMSOL AB. *COMSOL Multiphysics®*. Version 6.4. Stockholm, Sweden. URL: https://www.comsol.com.

[5] PhysicsNeMo Contributors. *NVIDIA PhysicsNeMo: An open-source framework for physics-based deep learning in science and engineering*. URL: https://github.com/NVIDIA/physicsnemo.

[6] Yanbo Fan et al. *Learning with Average Top-k Loss*. 2017. arXiv: 1705.08826 [stat.ML]. URL: https://arxiv.org/abs/1705.08826.

[7] A. Galántai. “The theory of Newton’s method”. In: *Journal of Computational and Applied Mathematics* 124.1 (2000). Numerical Analysis 2000. Vol. IV: Optimization and Nonlinear Equations, pp. 25–44. ISSN: 0377-0427. DOI: https://doi.org/10.1016/S0377-0427(00)00435-0. URL: https://www.sciencedirect.com/science/article/pii/S0377042700004350.

[8] Philip M. Gresho and Robert L. Sani. “On pressure boundary conditions for the incompressible Navier-Stokes equations”. In: *International Journal for Numerical Methods in Fluids* 7.10 (1987), pp. 1111–1145. DOI: https://doi.org/10.1002/fld.1650071008. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/fld.1650071008. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/fld.1650071008.

[9] Thomas J. R. Hughes. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Mineola, New York: Dover Publications, 2000. ISBN: 9780486411811.

[10] R. E. Hunt. *Chapter 2: Poisson’s Equation*. Lecture Notes, Mathematical Methods II, Natural Sciences Tripos, Part IB. University of Cambridge. 2002. URL: https://www.damtp.cam.ac.uk/user/reh10/lectures/nst-mmii-chapter2.pdf.

[11] Th.v. Kármán. “On the mechanism of the drag a moving body experiences in a fluid”. In: *Progress in Aerospace Sciences* 59 (2013). Special issue: Theodore von Kármán, pp. 16–19. ISSN: 0376-0421. DOI: https://doi.org/10.1016/j.paerosci.2013.04.001. URL: https://www.sciencedirect.com/science/article/pii/S0376042113000201.

[12] Charlotte Kauppinen. *A Comparison Between Multilayer Perceptron and Graph Convolutional Networks to Improve Pseudo-Time Stepping Convergence*. Technical Report. COMSOL, 2025, pp. 1–28.

[13] Thomas N. Kipf and Max Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. 2017. arXiv: 1609.02907 [cs.LG]. URL: https://arxiv.org/abs/1609.02907.

[14] C. Klaij. “Verification of BDF Time-Integration Method with Variable Step Size”. In: Jan. 2021. DOI: 10.23967/wccm-eccomas.2020.357.

[15] Nikola Kovachki et al. “Neural operator: Learning maps between function spaces with applications to pdes”. In: *Journal of Machine Learning Research* 24.89 (2023), pp. 1–97.

[16] Jiahang Li et al. *On the Initialization of Graph Neural Networks*. 2023. arXiv: 2312.02622 [cs.LG]. URL: https://arxiv.org/abs/2312.02622.

[17] Shirong Li and Xinlong Feng. “Dynamic Weight Strategy of Physics-Informed Neural Networks for the 2D Navier–Stokes Equations”. In: *Entropy* 24 (Sept. 2022), p. 1254. DOI: 10.3390/e24091254.

[18] Tianyu Li et al. *A fully differentiable GNN-based PDE Solver: With Applications to Poisson and Navier-Stokes Equations*. 2024. arXiv: 2405.04466 [physics.flu-dyn]. URL: https://arxiv.org/abs/2405.04466.

[19] Y. Li, T. Liu, and Y. Xie. "Thermal Fluid Fields Reconstruction for Nanofluids Convection Based on Physics-Informed Deep Learning". In: *Scientific Reports* 12.1 (2022), p. 12567. DOI: 10.1038/s41598-022-16463-1.

[20] NVIDIA. *MeshGraphNet: A Practical User Tutorial*. URL: https://docs.nvidia.com/physicsnemo/latest/user-guide/model_architecture/meshgraphnet.html.

[21] Jaemin Oh et al. *Spectrally Safe Neural Operator Warm-Starts for Large-Scale Newton Solvers*. 2026. arXiv: 2606.21828 [math.NA]. URL: https://arxiv.org/abs/2606.21828.

[22] Michael L. Overton. *Quadratic Convergence of Newton's Method*. Lecture notes, Numerical Computing, Spring 2017. 2017. URL: https://cs.nyu.edu/~overton/NumericalComputing/newton.pdf.

[23] Tobias Pfaff et al. *Learning Mesh-Based Simulation with Graph Networks*. 2021. arXiv: 2010.03409 [cs.LG]. URL: https://arxiv.org/abs/2010.03409.

[24] M. Raissi, P. Perdikaris, and G.E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2018.10.045. URL: https://www.sciencedirect.com/science/article/pii/S0021999118307125.

[25] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. *A Survey on Oversmoothing in Graph Neural Networks*. 2023. arXiv: 2303.10993 [cs.LG]. URL: https://arxiv.org/abs/2303.10993.

[26] Shaeke Salman and Xiuwen Liu. *Overfitting Mechanism and Avoidance in Deep Neural Networks*. 2019. arXiv: 1901.06566 [cs.LG]. URL: https://arxiv.org/abs/1901.06566.

[27] Michael Schäfer and Stefan Turek. "Benchmark Computations of Laminar Flow Around a Cylinder". In: *Flow Simulation with High-Performance Computers II*. Ed. by Ernst Heinrich Hirschel. Vol. 52. Notes on Numerical Fluid Mechanics. Vieweg, 1996, pp. 547–566.

[28] Robin Schmöcker et al. *Generalization capabilities of MeshGraphNets to unseen geometries for fluid dynamics*. 2024. arXiv: 2408.06101 [cs.LG]. URL: https://arxiv.org/abs/2408.06101.

[29] O. C. Zienkiewicz, R. L. Taylor, and J. Z. Zhu. *The Finite Element Method: Its Basis and Fundamentals*. 6th ed. Oxford: Elsevier Butterworth-Heinemann, 2005.

A

COMSOL configuration details

A.0.1. Time-Dependent Solver

The reference flow solutions used to generate the training and evaluation datasets were obtained using the time-dependent solver available in COMSOL Multiphysics. Time integration was performed using an implicit Backward Differentiation Formula (BDF) scheme. A second-order BDF method was used with a fixed time step of $\Delta t = 0.02\ \mathrm{s}$.

At each time step, the resulting nonlinear system was solved using a Newton method with a constant damping factor. The Jacobian matrix was updated once per time step. The main solver settings used throughout the simulations are summarized in Table A.1.

Table A.1: Time-dependent solver settings used for the numerical simulations.

Category	Parameter	Setting
Time integration	Solver type	Implicit
	Time integration method	BDF
	BDF order	2
	Time step	$0.02\ \mathrm{s}$
Time stepping	Steps taken by solver	Manual
	Initial step fraction	1
	Initial step growth rate	1.5
	Nonlinear controller	Enabled
Nonlinear solver	Nonlinear method	Constant (Newton)
	Damping factor	0.9
	Jacobian update	Once per time step
	Maximum number of iterations	8
Convergence	Termination technique	Tolerance
	Termination criterion	Solution
	Tolerance factor	0.5
Algebraic settings	Linear solver	Direct, fluid flow variables (spf)
	Singular mass matrix	Maybe
	Consistent initialization	Off
Error estimation	Algebraic variables	Excluded
Stabilization	Stabilization and acceleration	None

The stationary solver was settled exacty in the same way for both initializations, therefore eventual variations in the number on Newton iterations could be dependent just on the GNN initialization. Also the scaling and the residual scaling are both manually settled to 1 in the same sense. The maximum number of iterations was set to 100, to avoid that one of the solutions would not converge.

Table A.2: Stationary solver settings used for the stationary simulations.

Category	**Parameter**	**Setting**
Linear solver	Linear solver	Direct, fluid flow variables (spf)
Nonlinear solver	Nonlinear method Damping factor Jacobian update Automatic scale factors in weights	Constant (Newton) 1 On every iteration Enabled
Convergence	Termination technique Maximum number of iterations Tolerance factor Termination criterion	Tolerance 100 1 Solution
Linear solver tolerance	Adaptive tolerance	Disabled
Scaling	Method Scale	Manual 1
Residual scaling	Method Scale	Manual 1
Stabilization	Stabilization and acceleration	None

B

Configuration Hyperparameters

Table B.1: Hyperparameter configuration used for the first multi-Reynolds-number and multi-geometry training.

Component	Hyperparameter	Value
Training	Epochs	520
	Batch size	2
	Loss	MSE + top-k
	Top-k fraction / weight	0.10 / 10
	LR scheduler	StepLR
	Step size / γ	150 / 0.8
Velocity network	Architecture	MeshGraphNet
	Hidden dimension	64
	Learning rate	5×10^{-5}
	Processor steps	10
	Encoder layers (node / edge)	2 / 2
	Processor layers (node / edge)	2 / 2
	Decoder layers (node)	2
	Aggregation	Sum
Pressure network	Architecture	BSMS
	Hidden dimension	128
	Learning rate	10^{-5}
	Processor steps	10
	Encoder layers (node / edge)	2 / 2
	Processor layers (node / edge)	2 / 2
	Decoder layers (node)	2
	Mesh levels	5
	Aggregation	Sum

Table B.2: Hyperparameters optimized with Optuna and corresponding search spaces.

Hyperparameter	Search range	Distribution	Baseline value
Velocity learning rate	$[10^{-5}, 10^{-4}]$	Log-uniform	5×10^{-5}
Velocity hidden dimension	$\{64, \ldots, 96\}$	Integer uniform	64
Velocity processor steps	$\{10, \ldots, 15\}$	Integer uniform	10

Table B.3: Hyperparameter configuration used for the Jacobian scaled multi-Reynolds-number and multi-geometry training.

Component	Hyperparameter	Value
Training	Epochs	520
	Batch size	2
	Loss	Jacobian + Jacobian top-k
	Top-k fraction / weight	0.10 / 10
	LR scheduler	StepLR
	Step size / γ	150 / 0.8
Velocity network	Architecture	MeshGraphNet
	Hidden dimension	64
	Learning rate	5×10^{-5}
	Processor steps	10
	Encoder layers (node / edge)	2 / 2
	Processor layers (node / edge)	2 / 2
	Decoder layers (node)	2
	Aggregation	Sum
Pressure network	Architecture	BSMS
	Hidden dimension	128
	Learning rate	10^{-5}
	Processor steps	10
	Encoder layers (node / edge)	2 / 2
	Processor layers (node / edge)	2 / 2
	Decoder layers (node)	2
	Mesh levels	5
	Aggregation	Sum